

Взаимосвързани заявки

Необвързани заявки – когато външната и вътрешната заявка не са зависими една от друга по никакъв начин (точно такива сме правили досега).

Т.е. вътрешния SELECT не използва външния SELECT.

Условието, обикновено е просто, като: намерете най-високия връх в дадена планина, най-дългата река, най-високата заплата.

1 зад: Намерете най-високият връх:

```
-- най-високият връх:
• SELECT *
  FROM peaks
  WHERE elevation =
    (SELECT elevation FROM peaks
     ORDER BY elevation DESC
     LIMIT 1);
```

id	peak_name	elevation	mountain_id
68	Everest	8848	9
NULL	NULL	NULL	NULL

2 зад: Намерете най-високата заплата и служителят, който я получава

```
-- най-високата заплата и служителят, които я получава
• SELECT first_name, last_name, department_id, salary
  FROM employees WHERE salary =
    (SELECT salary FROM employees
     ORDER BY salary DESC
     LIMIT 1);
```

Резултат:

first_name	last_name	department_id	salary
Ken	Sanchez	16	125500.0000

3 зад: Намерете най-дългата река.

Обвързани заявки – вътрешния SELECT по някакъв начин зависи от външния SELECT.

Ако имаме външна и вътрешна заявка и искаме да наложим ограничение, така че една колона от външната заявка да се използва във вътрешната заявка, задаваме псевдоним на външната (обикновено това е началната буква на колоната) и използваме в WHERE клаузата във вътрешната. Така се получава, все едно работим с 1 таблица и нейно виртуално копие. Понеже, резултатът винаги е списък с повече от 1 резултати, накрая винаги извън подзаявката, като последен добавяме ORDER BY.

Например:

4 зад: Искаме да намерим най-дългата река, която се влива във всеки отделен водоем. За целта трябва да намерим най-дългата река (нещо, което можем и сме свикнали) в подзаявката, а после, да зададем повторението на колоната – в нашия случай, водоема. Т.е. намираме най-дългата река:

```
SELECT length FROM rivers
ORDER BY length DESC
LIMIT 1;
```

Обаче, искаме да изредим за всяко устие outflow поотделно, т.е. проверяваме 1 устие, намираме за него най-дългата река и преминаваме на следващо устие. И така изреждаме всички устия. Т.е. за колоната outflow ще имаме много повторения и много резултати. Тя трябва да се повтаря по някакъв начин. Тя прави връзка сама към себе си. Ако се намираме в различни таблици, ще имаме

WHERE таблица1.устие = таблица2.устие

Но ние се намираме в 1 и съща таблица rivers. Как да направим това повторение? Ами с виртуално копие на таблицата – едната таблица ще е rivers, а другата ще бъде нейният псевдоним rivers AS r. Това е нещо като виртуално копие на самата таблица. При 2 и повече таблици е много удобно, защото, вместо да се пише

WHERE таблица1.външен_ключ_id = таблица1.първичен_ключ_id

задаваме псевдоним с 1 буква (първата буква на таблицата) и вместо всеки път да пишем пълното име на таблицата, пишем само нейната първа буква.

Примерно WHERE peaks.mountaid = mountains.mountain_id

За да изброим планините, ще зададем псевдоними на таблиците и ще повторим планините:

peaks AS p

mountains AS m

...

WHERE p.mountain_id = m.mountain_id

- Само отбелязвам, че примерът е измислен за нагледност, а нашата таблица geography е по-сложна.

```
-- най-дългата река във всеки водоем
SELECT river_name, length, outflow
FROM rivers AS r      1 - псевдоним
WHERE length =
  (SELECT length FROM rivers  2 - повторение
   WHERE outflow = r.outflow -- повторението
   ORDER BY length DESC
   LIMIT 1)
ORDER BY outflow;      3 - подредба
```

Резултат:

river_name	length	outflow
Madeira	3380	Amazon
Salween	3060	Andaman Sea
Indus	3180	Arabian Sea
Syr Darya	3078	Aral Sea
Amazon	6400	Atlantic Ocean
Tocantins	3650	Atlantic Ocean, Amazon
Mackenzie	4241	Beaufort Sea
Yukon	3185	Bering Sea
Danube	2888	Black Sea

5 зад:

```
-- най-високият връх за всяка планина
SELECT *
FROM peaks AS p      задаваме псевдоним
WHERE elevation =
  (SELECT elevation FROM peaks      за да може вътре
   WHERE mountain_id = p.mountain_id да се повтори -- планина по веднъж
   ORDER BY elevation DESC
   LIMIT 1)
ORDER BY mountain_id;
```

Без реда на WHERE, изкарва само най-дългата река, но не за всяка група поотделно, няма изреждане, няма цикличност. С WHERE намираме най-голямата дължина, влагаме я със скоби в подзаयвка, после правим циклично изреждане на устията.

```
(SELECT length FROM rivers
WHERE outflow = r.outflow -- повторението
ORDER BY length DESC
LIMIT 1)
```

Резултат:

id	peak_name	elevation	mountain_id
79	Mount McKinley	6194	1
65	Damavand	5610	2
62	Aconcagua	6962	3
63	Botev	2376	4
67	Elbrus	5642	5
89	Vinson Massif	4897	7
81	Mount Sidley	4285	8
68	Everest	8848	9
69	Julianatop	4760	10

6 зад: Искаме да намерим най-високата заплата, но на всеки отдел поотделно, за целта, трябва да преглеждаме колона department_id и търсим само в нея.

```
-- най-високата заплата от всеки отдел и служителя, който я получава
• SELECT first_name, last_name, department_id, salary
  FROM employees AS e
 WHERE Salary =
  (SELECT salary FROM employees
   WHERE department_id = e.department_id -- отдел по веднъж
   ORDER BY salary DESC
   LIMIT 1)
 ORDER BY department_id;
```

first_name	last_name	department_id	salary
Terri	Duffy	1	63500.0000
Rob	Walters	2	29800.0000
Brian	Welcker	3	72100.0000
Sariya	Harnpadoungsataya	4	14400.0000
Mary	Gibson	4	14400.0000
Jill	Williams	4	14400.0000
Terry	Eminhizer	4	14400.0000
John	Wood	4	14400.0000

Оператори EXISTS / NOT EXISTS – използвали сме ги досега като

CREATE DATABASE IF NOT EXISTS,

DROP DATABASE IF EXISTS,

CREATE TABLE IF NOT EXISTS,

DROP TABLE IF EXISTS,

EXISTS е вярно ако тя връща записи

NOT EXISTS е вярно ако тя е празна

Те могат и да се вкарат вътре във взаимосвързана заявка.

Досега сме решавали най-много прости заявки в 1 таблица, ако знаем id на отдела. Можем да зададем псевдоним на таблицата с AS – обикновено се избира първата буква на таблицата. Оттам нататък, всеки път, вместо да пишем цялото име на таблицата, използваме само нейната (първа) буква.

```
-- Изведете всички служители от отдел финанси, ако знаем id:
SELECT first_name, last_name, department_id, salary
  FROM employees AS e
 WHERE e.department_id = 10;
```

Резултат:

first_name	first_name	department_id	salary
Deborah	Deborah	10	19000.0000
Wendy	Wendy	10	43300.0000
Candy	Candy	10	19000.0000
David	David	10	13500.0000
Bryan	Bryan	10	19000.0000
David	David	10	34700.0000
Dragan	Dragan	10	19000.0000

За 1 таблица няма никакъв смисъл, но ако са 2 или повече, е много по-удобно, вместо да пишем цялото име.

Нека не знаем id, съответно ще го използваме като връзка:

```
-- Изведете всички служители от отдел финанси, ако не знаем id:
-- тогава трябва да използваме id като връзка между 2 таблици:
-- employee AS е и departments AS d
-- и да погледнем името във втората таблица:
SELECT first_name, first_name, department_id, salary
FROM employees AS e WHERE EXISTS AS кодът ще тръгне и
( SELECT d.department_id FROM departments d без AS, но е по-
  WHERE e.department_id = d.department_id правилно да се
  AND d.name = 'Finance' ); напише
```

Може изобщо без псевдоними.

```
-- Изведете всички служители от отдел финанси, ако не знаем id:
-- тогава трябва да използваме id като връзка между 2 таблици:
-- employee AS е и departments AS d
-- и да погледнем името във втората таблица:
SELECT first_name, first_name, department_id, salary
FROM employees AS e WHERE EXISTS
( SELECT d.department_id FROM departments AS d -- влизам във втора таблица
  WHERE e.department_id = d.department_id -- id_първа = id_втора (връзка)
  AND d.name = 'Finance' ); -- името от втората
```

Резултат:

first_name	first_name	department_id	salary
Deborah	Deborah	10	19000.0000
Wendy	Wendy	10	43300.0000
Candy	Candy	10	19000.0000
David	David	10	13500.0000
Bryan	Bryan	10	19000.0000
David	David	10	34700.0000
Dragan	Dragan	10	19000.0000
Barbara	Barbara	10	26400.0000

Оператор NOT EXISTS:

-- Тримата най-високоплатени служители извън отдел Финанси:

- `SELECT first_name, first_name, department_id, salary`
`FROM employees e WHERE NOT EXISTS`

(
`SELECT department_id FROM departments d`
`WHERE e.department_id = d.department_id`
`AND d.name = 'Finance'`
)
`ORDER BY salary DESC LIMIT 3;`

first_name	first_name	department_id	salary
Ken	Ken	16	125500.0000
James	James	7	84100.0000
Brian	Brian	3	72100.0000